Fatec
São Caetano do Sul
.
Segurança da Informação

Ferramentas de Desenvolvimento para Web

Aula#2
Conceitos Fundamentais de Web

Professor: Veríssimo - carlos.pereira70@fatec.sp.gov.br

20/02/2026

Sobre este documento

Este documento objetiva deixar registrado o conteúdo abordado em sala de aula pelo professor. Importante destacar que a Nota de Aula serve como guia ao professor, bem como serve aos alunos como um norte, quanto ao conteúdo desenvolvido em sala de aula.
Este documento não tem a pretenção de ser uma única fonte para estudo. Para tal, o aluno deverá assistir às aulas e fazer uso (consulta) à bibliografia recomendada na ementa da disciplina, e à bibliografia complementar, apontada pelo professor.

Preâmbulo da Aula

A World Wide Web deixou de ser apenas um repositório de documentos interligados para se tornar a infraestrutura vital da sociedade moderna. Desde a simplicidade estática das páginas em HTML puro até a complexidade da Web Semântica e os horizontes da inteligência onipresente, a arquitetura por trás de cada clique evoluiu drasticamente.

Este documento explora os pilares fundamentais dessa evolução. Começaremos pela análise do modelo cliente-servidor e as fases históricas da Web (1.0 a 3.0), avançando para o futuro da Web Simbiótica (4.0). Além da cronologia, investigaremos as entranhas técnicas: o funcionamento dos motores de renderização, a abstração da infraestrutura física para a nuvem, o papel crucial das APIs e a anatomia dos protocolos que regem a comunicação global. O objetivo é fornecer uma visão holística que una o rigor técnico à compreensão ética dos desafios digitais contemporâneos.

Contents

0.1 Arquitetura Web e o Modelo Cliente-Servidor . 1

0.2 A Evolução da Web: Da Estática ao Futuro Inteligente 1

0.2.1 Web 1.0: A Era da Leitura (1989 – 2000) 1

0.2.2 Web 2.0: A Era da Interação (2000 – 2010) 2

0.2.3 Web 3.0: A Era da Semântica e Descentralização (2010 – Presente) 2

0.2.4 Comparativo Tecnológico 3

0.3 O Futuro: Web 4.0 e Além 4

0.3.1 Pilares da Web Simbiótica 4

0.3.2 Comparativo de Paradigmas: O Salto para o Futuro 5

0.4 Considerações Éticas e Desafios 6

0.5 O Ecossistema: Motores e Servidores 7

0.5.1 Motores de Renderização (Browsers Engines) 7

0.5.2 Servidores e Infraestrutura de Entrega . 8

0.6 Arquiteturas de Servidor: Do Físico ao Abstrato 9

0.7 O Papel das APIs e Microsserviços 11

0.7.1 Ciclo de Vida de um Clique (O Caminho do Pacote) 11

0.8 O Protocolo HTTP/HTTPS 12
0.8.1 Anatomia da Requisição (Request) . . . 12
0.8.2 Anatomia da Resposta (Response) . . . 13
0.8.3 O Problema da Memória: Statelessness . 13
0.9 Desafio Prático em Aula 14

0.1 Arquitetura Web e o Modelo Cliente-Servidor

A Web não é uma nuvem mística; é uma infraestrutura massiva de computadores conversando através de regras rígidas. Para o profissional de TI, a Web é um sistema distribuído de troca de recursos.

0.2 A Evolução da Web: Da Estática ao Futuro Inteligente

A World Wide Web não é apenas um emaranhado de cabos e servidores; é o tecido conectivo da sociedade moderna. Desde a sua concepção em um laboratório de física até a promessa de uma rede descentralizada, a Web passou por metamorfoses profundas que alteraram a forma como consumimos informação e nos relacionamos digitalmente.

—

0.2.1 Web 1.0: A Era da Leitura (1989 – 2000)

Tudo começou com **Sir Tim Berners-Lee**[1] no CERN. A proposta original era permitir que cientistas compartilhassem dados de forma eficiente através do hipertexto. Esta fase é fre-

[1] Tim Berners-Lee é o físico e cientista da computação britânico amplamente reconhecido como o inventor da World Wide Web (WWW). Em 1989, enquanto trabalhava no CERN (Organização Europeia para a Investigação Nuclear).Em 1994, ele fundou o World Wide Web Consortium (W3C), organização que lidera até hoje para garantir o desenvolvimento contínuo de padrões para a rede

quentemente chamada de "Web de leitura".

- **Características:** Páginas estáticas, uso de *frames* e tabelas para layout.
- **Tecnologia:** Surgimento do protocolo $HTTP$ e da linguagem $HTML$ primitiva.
- **Experiência do Usuário:** O papel do usuário era puramente passivo, sem interatividade ou geração de conteúdo próprio.

0.2.2 Web 2.0: A Era da Interação (2000 – 2010)

A virada do milênio trouxe a "Web Social". A internet deixou de ser um mural de avisos para se tornar uma praça pública, onde o conteúdo passou a ser gerado pelo usuário (*User Generated Content*).

- **Marcos:** O nascimento das redes sociais (Facebook, YouTube) e da Wikipédia.
- **Evolução Tecnológica:** Introdução do $AJAX$ (Asynchronous JavaScript and XML), permitindo atualizações parciais de páginas, e o avanço das $APIs$.
- **Mobilidade:** A internet saiu dos desktops para os smartphones, exigindo o desenvolvimento de design responsivo.

0.2.3 Web 3.0: A Era da Semântica e Descentralização (2010 – Presente)

Estamos vivendo a transição para uma rede mais inteligente, centrada no significado dos dados e na soberania do usuário.

1. **Inteligência Artificial:** Algoritmos que compreendem o contexto semântico das buscas.

2. **Blockchain:** A base para a descentralização, permitindo a propriedade real de ativos digitais sem intermediários.

3. **Interoperabilidade:** Diferentes ecossistemas digitais capazes de trocar informações de forma fluida.

0.2.4 Comparativo Tecnológico

A Tabela 1 resume a transição das pilastras técnicas que sustentam a Web, destacando a mudança de paradigmas ao longo das décadas.

Table 1: Evolução Cronológica das Tecnologias Base da Web.

Era	**Linguagens / Ferramentas**	**Paradigma de Design**
Primórdios	HTML puro, HTTP/0.9, Browsers de texto.	Foco em hiperlinks e exibição de documentos estáticos.
Anos 2000	PHP, MySQL, Flash, AJAX, CSS2.	Conteúdo dinâmico, portais e início da interatividade.
Anos 2010	HTML5, CSS3, React, Node.js, REST APIs.	Prioridade ao *Mobile-First*, Single Page Apps e Cloud.
Anos 2020+	Rust, Solidity, WebAssembly, IA Generativa.	Descentralização (Web3), IA integrada e borda (*Edge*).

0.3 O Futuro: Web 4.0 e Além

Se a Web 3.0 foca na descentralização e na propriedade dos dados, a **Web 4.0** — frequentemente chamada de *Web Simbiótica* ou *Web Ubíqua* — projeta um cenário onde a fronteira entre o físico e o digital torna-se praticamente invisível. Nesta etapa, a rede deixa de ser uma ferramenta que acessamos para se tornar um ambiente onde coexistimos.

0.3.1 Pilares da Web Simbiótica

O avanço para além da Web 3.0 é impulsionado por tecnologias que permitem uma interação em tempo real e de baixa latência, integrando a inteligência artificial de forma nativa ao cotidiano.

- **Onipresença (IoT):** A conexão de bilhões de dispositivos que tomam decisões autônomas através de *Edge Computing*.
- **Interfaces Cérebro-Computador (BCI):** A evolução das interfaces de usuário (UI) para além das telas, permitindo comandos por voz, gestos e, eventualmente, impulsos neurais.
- **Gêmeos Digitais (Digital Twins):** Representações virtuais precisas de cidades, fábricas e organismos vivos que interagem com a rede para simulações e otimizações.

0.3.2 Comparativo de Paradigmas: O Salto para o Futuro

A Tabela 2 detalha as mudanças fundamentais que se esperam na transição da rede atual para a Web 4.0.

Table 2: Comparativo entre a Web 3.0 e a Visão da Web 4.0.

Atributo	**Web 3.0 (Atual)**	**Web 4.0 (Futuro)**
Interação	Centrada em dados e semântica.	Centrada na simbiose humano-máquina.
Interface	Telas, realidade virtual e aumentada.	Interfaces imersivas e neuro-integradas.
IA	IA como assistente ou criadora de conteúdo.	IA como agente autônomo onipresente.
Conectividade	Alta velocidade (5G) e Cloud.	Ultra latência ($6G$) e Edge inteligente.
Objetivo	Descentralização e privacidade.	Eficiência global e automação total.

0.4 Considerações Éticas e Desafios

Com a Web 4.0, surgem desafios sem precedentes. A **privacidade cognitiva** torna-se um tema central, uma vez que a rede poderá processar não apenas o que fazemos, mas como reagimos biologicamente aos estímulos digitais. Além disso, a soberania da inteligência artificial sobre sistemas críticos exigirá novos frameworks de governança algorítmica.

A trajetória da Web demonstra que ela não é estática; ela é um reflexo da evolução da ambição humana. Se o passado foi sobre conectar documentos e o presente é sobre conectar pessoas, o futuro será sobre conectar a própria consciência à infraestrutura global de informação.

> **"A Web como eu a imaginei, ainda não a vimos. O futuro é muito maior do que o passado."**
> — ***Tim Berners-Lee***

0.5 O Ecossistema: Motores e Servidores

Para que a interface da Web chegue ao usuário final, existe uma complexa infraestrutura de *software* e *hardware* operando nos bastidores. Este ecossistema é dividido fundamentalmente entre os motores de renderização (*Client-side*) e os servidores de aplicação (*Server-side*).

0.5.1 Motores de Renderização (Browsers Engines)

O "motor" é o coração do navegador. Sua função é traduzir o código bruto ($HTML, CSS, JavaScript$) em uma interface visual interativa. A evolução desses motores determinou a velocidade e a conformidade com os padrões da $W3C$.

Chromium / Blink:
Mantido pela Google, é a base do Chrome, Edge e Opera. Foca em performance extrema e suporte rápido a novas APIs.

WebKit:
Motor nativo da Apple (Safari). Prioriza eficiência energética e integração com o ecossistema iOS/macOS.

Gecko:
Desenvolvido pela Mozilla (Firefox). É o principal defensor da web aberta e da privacidade, mantendo uma arquitetura independente.

0.5.2 Servidores e Infraestrutura de Entrega

Se o motor processa a informação, o servidor é quem a armazena e distribui. A evolução saiu de máquinas físicas locais para clusters globais de computação em nuvem (*CloudComputing*).

- **Servidores HTTP:** Softwares como *Nginx* e *Apache* gerenciam as requisições, garantindo que o conteúdo certo chegue ao IP correto.
- **CDNs (Content Delivery Networks):** Redes de servidores distribuídos geograficamente (como Cloudflare ou Akamai) que entregam o conteúdo a partir do nó mais próximo do usuário, reduzindo a latência.
- **Runtime Environments:** O surgimento do *Node.js* e *Deno* permitiu que o JavaScript saísse do navegador e passasse a rodar também no servidor, unificando a linguagem de desenvolvimento.

0.6 Arquiteturas de Servidor: Do Físico ao Abstrato

A escolha da arquitetura de servidor define não apenas o custo de um projeto, mas sua capacidade de sobrevivência a picos de tráfego e a complexidade de manutenção da equipe de *DevOps*.

Hospedagem Dedicada e VPS (Virtual Private Server)

Na hospedagem dedicada, o desenvolvedor possui controle total sobre o *hardware*. Já na **VPS**, um servidor físico é fatiado virtualmente.

- **Vantagem:** Controle granular sobre configurações do Kernel, sistema operacional e segurança.
- **Desafio:** O "desperdício de recursos" (pagar por 100% da CPU mesmo usando 5%) e a necessidade de intervenção humana para atualizações de segurança.

Arquitetura Serverless (Computação em Nuvem)

O termo *Serverless* é um nome comercial, pois os servidores ainda existem, mas são totalmente abstraídos pelo provedor (como AWS Lambda ou Google Cloud Functions).

- **Funcionamento:** O código é executado em resposta a eventos (triggers). Se ninguém acessa o site, nada é executado e o custo é zero.
- **Vantagem:** Escalabilidade infinita e automática. Se 1 milhão de usuários acessarem simultaneamente, o provedor provisiona instâncias instantâneas para suprir a demanda.

Análise Comparativa de Atributos

A Tabela 3 sintetiza as diferenças fundamentais entre os modelos, auxiliando na tomada de decisão arquitetural.

Table 3: Comparativo Técnico: Hospedagem Tradicional vs. Serverless.

Característica	**Hospedagem Dedicada / VPS**	**Arquitetura Serverless**
Gerenciamento	Gestão completa do SO, patches de segurança e *runtime*.	Abstração total; foco exclusivo na lógica do código (*Business Logic*).
Escalabilidade	Vertical (aumento de hardware) ou Horizontal manual.	Nativa e automática (*Auto-scaling* por requisição).
Modelo de Custo	Mensal fixo, independente do uso dos recursos.	*Pay-per-use* (paga-se apenas pelo tempo de execução).
Latência de Início	Baixa (o servidor está sempre "quente" e ligado).	Pode sofrer de *Cold Start* (atraso na primeira execução).
Casos de Uso	Aplicações legadas, bancos de dados complexos e tráfego estável.	APIs, processamento de imagens, tarefas assíncronas e *microsserviços*.

0.7 O Papel das APIs e Microsserviços

Atualmente, o ecossistema não é mais um "monolito" único. A Web moderna funciona como uma malha de microsserviços interconectados via **APIs RESTful** ou **GraphQL**. Isso permite que o motor do navegador busque dados de múltiplos servidores simultaneamente (ex: um servidor para autenticação, outro para o banco de dados e um terceiro para processamento de imagens), resultando em aplicações modulares e resilientes.

0.7.1 Ciclo de Vida de um Clique (O Caminho do Pacote)

1. **DNS Lookup:** O browser pergunta ao SO qual o IP do domínio.
2. **TCP Handshake:** Estabelecimento de conexão (SYN, SYN-ACK, ACK).
3. **SSL/TLS Handshake:** Se for HTTPS, as chaves criptográficas são trocadas aqui.
4. **HTTP Request:** O browser envia o pedido do recurso.
5. **Processamento:** O servidor processa (acessa banco de dados, executa scripts).
6. **HTTP Response:** O servidor devolve o HTML/Imagem/JSON.

Diferença Vital: Hospedagem vs Domínio

Domínio é o seu nome no RG (identidade). **Hospedagem** é o apartamento onde você mora (espaço em disco e CPU). Sem domínio, ninguém te acha; sem hospedagem, você não tem onde guardar suas coisas.

0.8 O Protocolo HTTP/HTTPS

O HTTP (*HyperText Transfer Protocol*) é um protocolo de camada de aplicação, base de toda a comunicação de dados na Web.

0.8.1 Anatomia da Requisição (Request)

Toda requisição que sai do cliente tem o seguinte formato:

Listing 1: Exemplo de Requisição HTTP

```
GET /index.html HTTP/1.1
Host: www.exemplo.com.br
User-Agent: Mozilla/5.0
Accept: text/html
```

Verbos (Métodos) Principais:

- `GET`: Busca um recurso (não deve ter corpo).
- `POST`: Envia dados para serem processados (ex: formulário).
- `PUT/PATCH`: Atualiza dados existentes.
- `DELETE`: Remove um recurso.

0.8.2 Anatomia da Resposta (Response)

O servidor responde com um código que dita o destino da aplicação:

- **2xx (Sucesso):** Ex: `200 OK`, `201 Created`.
- **3xx (Redirecionamento):** Ex: `301 Moved Permanently`.
- **4xx (Erro do Cliente):** Ex: `404 Not Found`, `403 Forbidden`.
- **5xx (Erro do Servidor):** Ex: `500 Internal Server Error`.

0.8.3 O Problema da Memória: Statelessness

O HTTP é **stateless** (sem estado). Ele esquece quem você é assim que a resposta é entregue. Para manter o usuário logado em um sistema da Fatec, por exemplo, usamos mecanismos de persistência:

Cookies vs Sessions

- **Cookies:** Pequenos arquivos de texto armazenados no **cliente** (navegador). Podem ser lidos pelo servidor em cada requisição.
- **Sessions:** Dados armazenados no **servidor**. O navegador guarda apenas um *Session ID* via Cookie para identificar qual "gaveta" no servidor pertence àquele usuário.

0.9 Desafio Prático em Aula

Abra o **Inspetor de Elementos** (F12) do seu navegador, vá na aba **Network** e acesse o site da Fatec.

1. Localize a primeira requisição. Qual o código de status?
2. Verifique nos *Request Headers* qual o seu `User-Agent`.
3. Tente encontrar algum Cookie sendo enviado na requisição.